MindBlown User Guide

MindBlown User Guide

Welcome. This guide walks you through planning a real project in MindBlown, from the first blank canvas to tracking progress, projecting a schedule, and running a daily standup. MindBlown's core idea is simple: the mindmap *is* your project plan. You brainstorm a tree, estimate the leaves, and everything above – effort totals, progress, health, schedules – computes itself. There is no "convert idea to task" step. A node starts as a label and gradually becomes a task as you add properties to it.

By the end of this guide you will know how to:

- Create a map and structure work as a mindmap using the keyboard
- Use AI brain-dump to turn prose into a starter tree
- Enrich a node into a task with estimates, dates, and a status
- Read the auto-computed progress rollups and health signals
- Set up dependencies, see a critical path, and project a finish date
- Organize releases with versions, milestones, and sprints
- Switch between mindmap, kanban, gantt, list, calendar, hill chart, and workload views
- Use the v0.7.0 AI features – brain-dump, breakdown, calibrated estimates, semantic search, and standup narratives

If you have not installed MindBlown yet, start with the README. For the "why" behind the tool, read the product vision.

---

1. Your first map

When you log in you land on the dashboard. It lists every map you have access to. To create a new one, click + **New Map** in the top right. A prompt appears asking for a name – type it and press Enter.

You land on an empty canvas with a single root node in the middle, already selected. That is your project. Rename the root by pressing `F2` or by double-clicking it.

Every map has an **effort unit** – `hours`, `days`, or `points`. `days` is the default and is a good fit for most software projects; `hours` suits short consulting engagements; `points` is there if you run a story-point workflow. You can change the effort unit per map from the workspace settings later. Values are not automatically reinterpreted when you switch – `5 hours` does not become `5 days`.

[screenshot: empty map with a single root node selected, toolbar on top, sidebar on right]

A few things about the canvas:

- Scroll to zoom. Drag on empty space to pan. `Cmd/Ctrl+0` fits the whole tree to the screen.
- Click a node to select it. Double-click to edit the label. Press `Escape` to stop editing.
- The sidebar on the right is the **Property Panel** – it shows whichever node is selected and is where you enrich nodes into tasks.

---

2. Brainstorming: getting ideas out of your head

There are two ways to turn a blank canvas into a project structure. Learn both. Start with whichever feels natural for the project in front of you.

2a. Manual: keyboard-first tree building

The editor is built for the keyboard. With the root node selected, press `Tab`. A new child node appears, already in edit mode. Type a label, press `Enter`, and a sibling is created below it. Press `Tab` again and that sibling gets a child. This is the entire rhythm of brainstorming in MindBlown:

- `Tab` – create a child under the selected node
- `Enter` – create a sibling (at the same level)
- `F2` – rename the selected node

- `Delete` / `Backspace` – remove the selected node (and any descendants)
- `Space` – collapse or expand the subtree below the selected node
- Arrow keys – move the selection left (parent), right (first child), up and down (siblings)
- `Cmd/Ctrl+A` – select all nodes
- `Cmd/Ctrl+0` – fit the map to the screen
- `Escape` – if you are drilled into a subtree, go back up one level; otherwise clear selection

A typical first pass: top-level branches for the functional areas of the project (“Design”, “Backend”, “Frontend”, “Launch”), then one more level of specifics under each. You are not trying to estimate anything yet. You are thinking out loud with structure.

[screenshot: a freshly brainstormed tree with 4 top-level branches, each with 2-3 children, radial layout]

Drag a node and it stays where you dropped it – freeform positioning is first-class. If you want to tidy up, pick a layout from the toolbar: radial, left-to-right, top-down, or org chart.

2b. AI brain-dump: prose to tree in one step

If you already have a description somewhere – a kickoff doc, meeting notes, a Slack thread, a feature brief – you do not have to retype it into nodes. Right-click the node you want the new subtree to hang off, and pick **AI Brain Dump** from the context menu.

A modal appears with a textarea. Paste your prose and click **Generate**. MindBlown sends the text to the configured language model and returns a nested tree, which replaces the textarea with a flat, indented preview of the proposed nodes. Every row is editable – tweak the label, adjust the estimate, or click the red × to drop a node you do not want. When you are happy, click **Create N nodes** and the subtree is created under the node you started from. Click **Cancel** and nothing changes.

Brain-dump is the fastest way to bootstrap a project from an existing artifact. It is not magic; expect to tidy the tree afterwards. Think of it as a motivated intern who takes notes for you.

[screenshot: brain-dump modal showing the indented preview with editable rows and the “Create N nodes” button]

---

3. From ideas to tasks: gradual enrichment

Not every node has to be a task. A brainstorming map with fifty nodes and zero estimates is still a useful artifact. But at some point, a handful of nodes graduate from “idea” to “real work I need to track.” Here is how.

Click the node. The **Property Panel** opens on the right. It has fields for:

- **Description** – rich text, the long-form body of the node. Every node is also a mini-page.
- **Status** – `To Do`, `In Progress`, `Done`, or `Blocked`.
- **Priority** – P0 through P3.
- **Assignee** – pick a workspace member.
- **Due date** – and optionally a start date, for duration-based planning.
- **Effort estimate** – see the next section.
- **Percent complete** – see section 6.
- **Version** / **Milestone** / **Sprint** – release planning tags, covered in section 9.

You do not need to fill any of these in to have a valid node. Add only what you need. The moment a node has a status, it starts showing up on the kanban board. Give it a due date and it appears on the calendar. Add an assignee and it counts toward that person’s workload. Each field you add unlocks a new way to view the same node – this is what we mean by “gradual enrichment.”

[screenshot: property panel with description, status dropdown, due date, assignee, and effort fields visible]

---

4. Estimating effort

This is the most important section in the guide. The way you estimate in MindBlown is a little different from every other PM tool, and getting it right is what makes the rest of the product work.

The leaf rule

Only leaf nodes get estimates. Parent nodes compute their estimate by summing their children. You never type an estimate into a parent. If a node has children, its effort number is derived, not stored.

This has two consequences:

1. **Breakdown is estimation.** If a node feels too big to guess at, split it into children. Estimate the children. The parent's total emerges. Splitting a 20-day "Redesign checkout" node into 5 child tasks you can each estimate more confidently is how you get honest numbers.
2. **Moving a leaf reshapes the plan.** Drag a leaf under a different parent and both the old and new parents recompute instantly. The tree is live.

The effort unit you picked when creating the map (days / hours / points) is the unit for every estimate in that map. You can change it later in map settings, but values are not automatically reinterpreted – `5 hours` does not become `5 days`.

Manual estimates

With a leaf selected, type a number into the **Estimate** field in the property panel. That is all.

AI-calibrated estimates

Next to the estimate field there is an **AI** button. Click it and MindBlown asks the configured model to estimate the work, using the node's label and description plus surrounding context. But there is a twist: before the suggestion is shown to you, it is multiplied by a **calibration factor** based on your own historical data.

Here is how that works. Over time, as you log actual effort on completed nodes, MindBlown builds a ratio of what you planned versus what it actually took. If on average your real efforts come in 1.4x over plan, the raw AI estimate is multiplied by 1.4 before you see it. This is velocity correction – it fights optimism bias without you having to think about it.

The suggestion is shown with three numbers: the **calibrated estimate** (what you should use), the **raw estimate** (what the model said), and a **sample count** – how many completed nodes the calibration is based on. If samples is 0, no calibration has happened yet and you are looking at the raw number. Trust it less. The more work you complete, the better the calibration gets.

Click **Accept** to apply the number, or close the dialog to ignore it. The AI never writes to your map without confirmation.

---

5. Tracking progress

Once estimates are in place, actual work happens. As each leaf moves along, update its **Percent complete** field (0 to 100) from the property panel.

You never update percent complete on a parent. Like estimates, parent progress is computed.

The weighted rollup

A parent's progress is the **effort-weighted average** of its children:

```
parent.progress = sum(child.estimate * child.progress) /
sum(child.estimate)
```

This is the right way to roll up progress, and it matters. Consider two sibling leaves:

Leaf	Estimate	Progress
A	2 days	50%
B	8 days	0%

A naive average says the parent is at 25% complete. But the real answer is:

```
(2 * 50 + 8 * 0) / (2 + 8) = 100 / 10 = 10%
```

The parent is **10% complete**, not 25%, because the 8-day task carries four times the weight of the 2-day task. That single leaf dominates the rollup. If you were judging the project by the unweighted number you would be wildly optimistic.

This weighting applies all the way to the root. Glance at the root node and you see the effort-weighted progress of the entire project in one number.

[screenshot: a small subtree showing child leaves with their estimates and progress, and the computed parent number visible on the parent node]

---

6. Health signals

Effort and progress give you numbers. Health signals tell you whether those numbers are *okay*.

Every node has a health state:

- **On track** – progress is roughly proportional to the time elapsed between now and the due date.
- **At risk** (amber) – progress is lagging. You are not yet in trouble, but you will be soon.
- **Behind** (red) – progress is significantly off what you would need to finish on time.

Health is computed from the due date, the elapsed share of that window, and the current percent complete. A leaf with a due date 10 days away, 5 days in, and at 20% complete is behind. A leaf with no due date is always "on track" – the signal is optional and opt-in.

Worst-child-wins propagation

Here is the part worth pinning to your monitor: **a parent's health equals the worst health of any descendant**. One leaf somewhere deep in the tree turning red drags the entire ancestor chain red, all the way to the root.

This sounds alarming. It is a feature. It means you do not have to go looking for problems. A single stuck task cannot hide in a subtree; it lights up its branch. Open the map, find the red or amber branches, drill in, and you are already looking at the problem.

The corollary: green means green. If your root node is green, every descendant is on track. You do not need to audit; the rollup already did.

[screenshot: a map where one leaf deep in the Backend branch is red, and the Backend branch and root are both amber/red]

---

7. Dependencies and schedule

Progress is one half of planning. The other half is *when*.

Creating a dependency

Dependencies have four types. Pick whichever models the real-world constraint:

Type	Meaning
FS	Finish-to-Start – B starts after A finishes (most common)
SS	Start-to-Start – B cannot start until A starts
FF	Finish-to-Finish – B cannot finish until A finishes
SF	Start-to-Finish – B cannot finish until A starts

As of v0.7.0 you create dependencies through the API or via the `add_dependency` MCP tool – there is not yet a direct gesture on the canvas. Once created, they render on the mindmap as dotted lines between nodes, and on the Gantt view as arrows between bars. The planning math (critical path, schedule projection) picks them up immediately. Removal is the same: `remove_dependency` via MCP or the API.

Critical path

Once you have a few dependencies in place, MindBlown walks the graph and highlights the **critical path** – the longest chain of dependent tasks that determines the earliest possible finish. Nodes on the critical path get a distinct visual marker. A one-day slip on a critical-path task slips the whole project by a day. A slip on a non-critical task absorbs into slack.

Schedule projection

The schedule computation takes estimates, dependencies, the project start date, and the effort-unit conversion (hours-per-day if you are in hours) and produces a projected timeline. The Gantt view is the visual expression of that computation. You can also pull it programmatically via the `get_schedule` MCP tool if you are driving MindBlown from an AI agent – see the MCP guide.

Change any input – extend an estimate, add a dependency, mark a blocker done – and the projection updates instantly. You are not running a scheduler; you are looking at one.

[screenshot: gantt view with critical-path tasks highlighted in red and dependency arrows between bars]

---

8. Versions, milestones, and sprints

So far we have organized work by *what* – the functional structure of the tree. Release planning is about *when*. MindBlown handles "when" as a separate layer, orthogonal to the tree. This is the single most important thing to understand about how MindBlown models release planning.

You do not move nodes to plan releases. You tag them.

There are three entities in the release layer, each with its own purpose.

Versions

A version is a **release container**, like `V1` or `2.0`. Create versions from the **Versions panel** in the sidebar – not by creating a node in the tree. A version has a name, a target date, and a status (planning, active, released, archived).

To put a node into a version, select the node and pick the version in the property panel. The node does *not* move in the tree. It just gains a tag. The same node can contribute to the functional area it lives in (say, "Compliance > Data Retention") *and* ship in V2.

Milestones

A milestone is a **key deliverable** within a version – "Billing module complete", "Public beta", "Kernsystem MVP". Milestones are first-class entities, not a boolean flag on a node. Create them from the Versions panel under their owning version. Give them a name, a target date, and link any nodes that contribute to the milestone.

Milestone progress is itself a weighted rollup over the linked nodes. On the Gantt view, milestones appear as diamonds.

Sprints (cycles)

A sprint is an **optional** time-boxed iteration within a version. One to four weeks is typical. Create sprints from the **Sprint panel**. Assign leaf nodes to a sprint from the property panel, or in bulk by multi-selecting leaves and using the bulk-assign action.

Everything in MindBlown works without sprints. If you are not a sprint team, ignore this layer entirely; the planning loop is complete without it. If you do run sprints, you get a sprint overview, rollover for unfinished items, and a sprint filter on every view.

Why the tags are orthogonal

Imagine a feature called “Invoice export”. In the tree it lives under `Compliance > Reporting > Exports`. That is where it belongs functionally – anyone looking for reporting work finds it there. Separately, it is tagged to `V1`, linked to the `Billing` milestone, and assigned to `Sprint 3`. None of those tags move the node in the tree. You can answer “what work is left in V1?” and “what does the Compliance area look like?” without those two views fighting over where the node lives.

---

9. Views

Everything we have covered so far happens on the mindmap – and the mindmap is always the primary surface. But the same data renders as several other views when you need a different angle. Switch with the view picker in the top toolbar. There is no sync step and no data transformation; it is the same nodes, re-rendered.

View	When to use it
Mindmap	Planning, restructuring, brainstorming – the primary surface
Kanban	Day-to-day task flow, grouped by status, priority, assignee, or sprint
Gantt	Timeline review, dependency chains, critical path, schedule conversations
List/Table	Spreadsheet-style bulk edits, sorting, filtering, and cross-cutting views
Calendar	“What is due this week” at a glance

View	When to use it
Hill Chart	Honest status without percent-complete theatre – are we figuring it out, or executing?
Workload	Who is overloaded, who has slack, who should take the next item

A node tagged into V1, assigned to Sprint 3, and living under `Compliance > Reporting` appears everywhere the moment you add the tag. You never file the same work in two places.

[screenshot: the view picker in the toolbar with all seven view options]

---

10. AI features (v0.7.0)

MindBlown's AI features are opt-in enhancements built on top of the core planning loop. Nothing below runs without you clicking a button, and nothing writes to your map without a confirmation step.

Brain-dump: prose to tree

Covered in section 2b. Paste prose, get a preview of a nested subtree, accept or discard. The fastest way to start a map from an existing document.

Break down a task

Pick a node that feels too big to estimate. Right-click it and choose **AI Breakdown** from the context menu. A modal opens asking how many subtasks you want (2 to 15, default 5) and an optional context hint – use that to nudge the model ("focus on backend", "skip tests", etc.). Click **Generate**.

MindBlown shows the suggestions as an editable list. Tweak any label, adjust any estimate, and click the × next to any suggestion you want to drop (click again to restore it). When you are happy, click **Accept N tasks** and they become real children of the node. Good for unblocking "I can tell this is big but I don't know where to start" moments.

Calibrated estimate

Covered in section 4. The AI estimate suggestion is multiplied by your historical planned-vs-actual ratio before being shown, so the number you see already accounts for your team's optimism bias. Samples = 0 means no calibration yet, so trust the number less.

Semantic search

Open the command palette with `Cmd/Ctrl+K` and start typing. Results now include **semantic matches** as well as substring matches. Searching "login" finds a node called "user authentication" even though the word "login" does not appear in it. The index is built from node labels and descriptions and updates in the background as you edit. Semantic search is especially useful on large maps where you remember the idea but not the exact wording.

AI standup

MindBlown can generate a three-section narrative over the last N hours of activity on a map:

- **Done** – what was completed or moved forward
- **In progress** – what is currently being worked on
- **Blockers** – anything flagged blocked, at risk, or behind

In v0.7.0 this is exposed through the API (`POST /api/ai/standup`) and the `ai_standup` MCP tool, but is not yet wired into the web UI. The intended use is daily standups – point an AI agent at your map over MCP, ask for a standup, and paste the output into your team chat. See the MCP guide for the tool call.

---

11. Collaboration

MindBlown is real-time. Multiple users can edit the same map at the same time, and their cursors appear on the canvas with their name and color so you can see where everyone is working. Changes from other users appear as they happen, without a refresh.

- **Comments** – every node has a comment thread. Open the Comments panel from the sidebar when a node is selected. You can edit and delete your own comments.

- **Sharing** – open the **Share dialog** from the toolbar. Invite people by email with one of three permission levels: **view**, **edit**, or **admin**. You can also generate a **public read-only link** for sharing with non-members – handy for showing a plan to a client without setting up accounts.
- **Presence** – cursors and selections from other users are visible as long as they are on the same map.

---

12. GitHub integration

MindBlown lives alongside your ticket system, not instead of it. Here is how to wire it up to a GitHub repo.

1. Open the **GitHub panel** from the sidebar.
2. Enter a GitHub personal access token, the repo **owner**, and the repo **name**, then click **Connect GitHub**. The status flips to "Connected to owner/repo" when it works.
3. You now have three operations available per node:
    - **Link to existing issue** – paste an issue URL or number, and the node starts syncing status, assignee, and labels with that issue.
    - **Create issue from node** – promote a leaf into a new GitHub issue. The label becomes the title; the description becomes the body; the assignee carries over.
    - **Import issues** – pull issues from the repo into the map as a flat list of nodes, which you then drag into the right branches.

When a linked issue is closed, the node's progress goes to 100% automatically and rolls up through its ancestors. When you move an issue between columns in a linked GitHub Project, the node's status follows. The integration is bidirectional but opinionated – MindBlown is the planning layer, GitHub is the execution layer.

---

13. Keyboard shortcuts reference

Shortcuts that apply on the mindmap canvas (when a node is selected and you are not editing its label):

Shortcut	Action
`Tab`	Create a child under the selected node
`Enter`	Create a sibling of the selected node

Shortcut	Action
`F2`	Rename the selected node
`Delete` / `Backspace`	Delete the selected node(s)
`Space`	Collapse or expand the subtree
`Escape`	Drill up one level, or clear selection
Arrow Left	Select parent
Arrow Right	Select first child
Arrow Up	Select previous sibling
Arrow Down	Select next sibling
`Cmd/Ctrl+A`	Select all nodes
`Cmd/Ctrl+0`	Fit the map to the screen

Global shortcuts (work anywhere in the app):

Shortcut	Action
`Cmd/Ctrl+K`	Open the command palette (semantic search + actions)
`Q`	Open quick-add from anywhere
`Escape`	Close any open dialog

Quick-add parses natural language: type `Design homepage due Friday p1 @sarah` and MindBlown creates a node with the label "Design homepage", a due date of Friday, priority P1, and Sarah as the assignee.

---

14. Where to go next

You have the loop. From here:

- MCP guide – expose your MindBlown maps to Claude, Cursor, or any MCP-capable AI agent. Covers all the tools, resources, and example conversations.
- API reference – REST endpoints for custom integrations and scripts.
- Self-hosting guide – deploy MindBlown on your own infrastructure.
- Product vision – the "why" behind the tool and the competitive landscape.
- README – feature catalog and quick start.

The planning loop in one sentence: **map the work, estimate the leaves, update progress, watch health propagate, let the schedule emerge.** Everything else is a view onto that loop. Have fun.